

Infrastructuur en Deployen

BlockyTest, low-code testautomatisering via visuele blokken

Naam: Amira El Hafidi

Team: Dragon Slayers

Datum: 29 mei 2026

Werkende URL: <https://blockytest.nl>

1. Inleiding

Mijn doel was om te leren hoe je een webapplicatie van een lokale ontwikkelomgeving naar een online omgeving brengt. Tot dat moment draaide BlockyTest alleen lokaal op mijn eigen computer. Dit betekende dat de applicatie niet bereikbaar was zodra mijn laptop uitstond. Daarom wilde ik de applicatie hosten op een server die continu online staat, zodat Wailsalutem BV de applicatie zelfstandig kan gebruiken.

Voor mij was dit een volledig nieuw onderwerp. Ik had nog niet eerder gewerkt met servers, Docker of domeinkoppelingen en wist ook niet hoe HTTPS moest worden ingesteld. Tijdens dit proces heb ik stap voor stap geleerd hoe een applicatie online beschikbaar wordt gemaakt en welke technieken daarvoor nodig zijn.

In dit verslag beschrijf ik mijn volledige leerproces: van het onderzoeken van verschillende hostingoplossingen tot het uiteindelijk succesvol deployen van de applicatie met een eigen domeinnaam en HTTPS-beveiliging. Daarbij beschrijf ik niet alleen de uiteindelijke oplossing, maar ook de opties die ik heb onderzocht en waarom bepaalde keuzes uiteindelijk niet zijn gebruikt. Hierdoor wordt duidelijk hoe de uiteindelijke deploy stack tot stand is gekomen.

2. Begrippen uitgelegd

Voordat ik de stappen beschrijf, leg ik kort uit wat de belangrijkste termen betekenen.

- **VPS (Virtual Private Server):** Een VPS is een virtuele server die via het internet op afstand beheerd kan worden. Het werkt vergelijkbaar met een eigen computer die altijd aan staat en continu verbonden is met het internet.
- **Docker:** Docker is software waarmee een applicatie wordt verpakt in een container. In zo'n container zitten alle onderdelen die nodig zijn om de applicatie correct te laten werken. Hierdoor werkt de applicatie op iedere omgeving hetzelfde.
- **Docker Compose:** Docker Compose maakt het mogelijk om meerdere containers tegelijk op te starten en samen te laten werken. In dit project wordt Docker Compose gebruikt om zowel de Flask-applicatie als de reverse proxy automatisch samen te starten.
- **Reverse Proxy:** Een reverse proxy functioneert als een tussenlaag tussen de bezoeker en de applicatie. Wanneer iemand naar blockytest.nl gaat, komt het verzoek eerst binnen bij de reverse proxy. Deze stuurt het verzoek vervolgens door naar de juiste applicatie op de server. Daarnaast zorgt de reverse proxy voor HTTPS-beveiliging, zodat de verbinding versleuteld is.
- **DNS (Domain Name System):** DNS is het systeem dat domeinnamen koppelt aan IP-adressen. Een IP-adres is lastig te onthouden, daarom gebruiken mensen

domeinnamen zoals blockytest.nl. DNS zorgt ervoor dat de domeinnaam automatisch wordt gekoppeld aan het juiste IP-adres.

- **A-record:** Een A-record is een DNS-instelling die een domeinnaam koppelt aan een specifiek IP-adres. Wanneer iemand blockytest.nl invoert in de browser, gebruikt DNS het A-record om te bepalen naar welke server het verzoek gestuurd moet worden.
- **SSH (Secure Shell):** SSH is een veilige methode om op afstand verbinding te maken met een server. Via de terminal kunnen commando's worden uitgevoerd op de server zonder dat fysieke toegang nodig is.
- **SSH-key:** Een SSH-key is een digitale sleutel die bestaat uit een publieke en een private key. De publieke key wordt opgeslagen op de server en de private key blijft op de eigen computer staan. Tijdens het inloggen controleert de server of beide sleutels bij elkaar horen. Hierdoor kan veilig worden ingelogd zonder wachtwoord.
- **HTTPS en SSL-certificaat:** HTTPS is de beveiligde versie van HTTP en zorgt ervoor dat gegevens versleuteld worden verzonden tussen de gebruiker en de website. Voor HTTPS is een SSL-certificaat nodig. In dit project worden de certificaten geleverd via Let's Encrypt, een organisatie die gratis SSL-certificaten aanbiedt.
- **CI/CD-pipeline:** CI/CD staat voor Continuous Integration en Continuous Deployment. Dit is een geautomatiseerd proces waarbij code na een wijziging automatisch wordt getest en gedeployed. Hierdoor hoeft niet handmatig op de server te worden ingelogd om updates door te voeren of de applicatie opnieuw op te starten.

3. Zoekproces naar de juiste hosting

Voordat ik de applicatie online kon zetten, moest ik eerst kiezen waar deze gehost zou worden. Dit bleek lastiger dan verwacht, omdat er verschillende mogelijkheden waren met ieder hun eigen voor- en nadelen. Daarom heb ik meerdere opties onderzocht en met elkaar vergeleken. Hieronder beschrijf ik welke oplossingen ik heb overwogen en waarom ik uiteindelijk heb gekozen voor TransIP.

DigitalOcean

Ik heb gekeken naar DigitalOcean, een populaire cloud provider die door veel ontwikkelaars wordt gebruikt.

Ik ben helemaal door het registratieproces gegaan en stond op het punt om een Droplet (zo noemen ze hun VPS) aan te maken in Amsterdam. Alles was geselecteerd, Ubuntu 22.04 en het juiste plan. Bij het afrekenen kwam ik er echter achter dat DigitalOcean alleen creditcards en PayPal accepteert. Geen iDEAL.

Aangezien ik geen creditcard heb, viel DigitalOcean af. Dit was teleurstellend omdat de service zelf er goed uitzag.

Strato

Vervolgens heb ik gekeken naar Strato. Dit is een Duitse provider met goede prijzen, ze hebben Linux VPS pakketten vanaf één euro per maand. Ze accepteren wel iDEAL en zijn populair in Nederland.

Het probleem bij Strato waren de eenmalige setupkosten van negen euro. Voor een schoolopdracht van één maand vond ik dat veel. De totale kosten zouden uitkomen op elf euro voor één maand gebruik, terwijl andere opties geen setupkosten hadden.

Hostinger

Hostinger was een serieuze optie. De website en het beheerpaneel zijn erg gebruiksvriendelijk en veel onderdelen kunnen met een paar klikken worden ingesteld. Dit maakt het platform vooral aantrekkelijk voor beginners. Daarnaast accepteert Hostinger iDEAL en waren de prijzen vergelijkbaar met andere aanbieders.

Toch heb ik uiteindelijk niet voor Hostinger gekozen. Tijdens mijn onderzoek merkte ik dat veel onderdelen sterk geautomatiseerd zijn en dat het platform veel voor de gebruiker probeert te regelen. Hoewel dit handig kan zijn, wilde ik juist zelf leren hoe het opzetten en beheren van een server werkt.

TransIP (uiteindelijke keuze)

Uiteindelijk heb ik gekozen voor TransIP. Dit is een Nederlandse provider met een goede reputatie en ondersteuning voor iDEAL. Daarnaast waren de domeinnamen goedkoop.

Een groot voordeel was dat ik zowel de VPS als de domeinnaam bij dezelfde provider kon regelen. Hierdoor stonden alle instellingen op één plek en werkte het instellen van DNS eenvoudig samen met de VPS.

Daarnaast gaf TransIP mij meer vrijheid om alles zelf in te stellen. Hierdoor kon ik beter leren hoe het deployen en beheren van een server werkt.

/ Factuur

Omschrijving	Termijn	Factureringsperiode	Aantal	Prijs
VPS V1 (aelhafidi-vps)	Maandelijks	24-05-2026 - 23-06-2026	1	€ 5,00
Subtotaal				€ 5,00
+ BTW (21%)				€ 1,05
Totaal:				€ 6,05

4. Zoekproces naar de juiste reverse proxy

Naast de hosting moest ik ook kiezen welke reverse proxy ik wilde gebruiken. Een reverse proxy zorgt ervoor dat bezoekers via mijn domeinnaam bij de juiste applicatie uitkomen. Er zijn meerdere opties beschikbaar en ik heb verschillende mogelijkheden vergeleken voordat ik mijn keuze maakte.

Apache

Apache is een van de oudste en bekendste programma's voor dit werk. Veel grote websites gebruiken het en daardoor is er online enorm veel informatie over te vinden. Het grote nadeel van Apache is dat je alles moet instellen via ingewikkelde tekstbestanden. Elke kleine wijziging moet je met de hand in de code van de server typen. Omdat ik voor het eerst met een server werk, vond ik dit te lastig en de kans op fouten te groot.

Caddy

Daarnaast heb ik gekeken naar Caddy. Dit is een moderner programma. Een heel groot voordeel van Caddy is dat het automatisch het veilige HTTPS-slotje regelt voor je website, zonder dat je daar extra tools voor nodig hebt. De code die je moet schrijven is ook veel korter dan bij Apache. Toch had Caddy voor mij één nadeel: het heeft geen visueel dashboard. Je moet alles nog steeds intypen in een tekstbestandje. Ik vond het toch een stuk fijner om een grafische omgeving te hebben waar ik gewoon met mijn muis in kan klikken.

Nginx Proxy Manager (uiteindelijke keuze)

Uiteindelijk heb ik gekozen voor Nginx Proxy Manager (NPM). Dit programma gebruikt op de achtergrond de webserver Nginx, maar voegt daar een gebruiksvriendelijk dashboard aan toe.

Dit programma biedt voor mij de volgende voordelen:

- **Eenvoudig beheer:** Via de browser kunnen eenvoudig nieuwe domeinnamen worden toegevoegd en met een paar klikken aan de juiste applicatie worden gekoppeld, zonder dat er code geschreven hoeft te worden.
- **Automatische beveiliging:** Met één klik kan een gratis beveiligingscertificaat (Let's Encrypt) worden aangevraagd. NPM zorgt er daarna zelf voor dat de website altijd via een beveiligde HTTPS-verbinding bereikbaar is.
- **Inzichtelijk:** Door het visuele dashboard is direct in één oogopslag te zien welke website aan welke applicatie is gekoppeld. Bij oplossingen zoals Apache of Caddy is specifieke technische kennis nodig om de configuratiebestanden te kunnen lezen en begrijpen.

5. Mijn definitieve deploy stack

Na het vergelijken van verschillende opties heb ik uiteindelijk gekozen voor de onderstaande deploy stack. Deze combinatie sluit goed aan op de opdracht en geeft mij voldoende controle om zelf te leren hoe deployment en serverbeheer werken.

Onderdeel	Keuze	Reden
Computer waar de app draait	VPS bij TransIP	Eigen controle, betalen met iDEAL, Nederlands bedrijf
Besturingssysteem	Ubuntu 22.04 LTS	Stabiele versie met veel documentatie online
Container technologie	Docker en Docker Compose	Standaard in de industrie, makkelijk te gebruiken
Reverse proxy	Nginx Proxy Manager	Visuele interface, automatische HTTPS via Let's Encrypt
Domeinnaam	blockytest.nl via TransIP	Slechts 59 cent het eerste jaar, zelfde provider als de VPS
CI/CD platform	GitLab CI/CD	Onze code staat al op GitLab

Deze stack combineert een Nederlandse VPS-provider, een eenvoudige reverse proxy en automatische deployment via GitLab CI/CD. Hierdoor kon de applicatie stabiel en veilig online worden gezet.

6. Stap voor stap deployen

Stap 1: VPS aanschaffen bij TransIP

Ik heb op de website van TransIP het VPS-pakket V1 besteld. Tijdens het bestellen heb ik gekozen voor Ubuntu 22.04 LTS als besturingssysteem.

Voor de inlogmethode kon ik kiezen tussen SSH-keys, een eenmalig wachtwoord of cloud-config. Ik heb gekozen voor SSH-keys, omdat dit veiliger is dan inloggen met een normaal wachtwoord.

Na de betaling via iDEAL ontving ik per e-mail de gegevens van de VPS, waaronder het IP-adres 37.97.220.7 en de gebruikersnaam aelhafidi. Vervolgens werd Ubuntu automatisch geïnstalleerd op de server. Dit duurde een paar minuten, waarna de VPS klaar was voor gebruik.



Stap 2: Inloggen op de VPS via SSH

Ik probeerde via PowerShell in te loggen op de server met SSH. Dit werkte in het begin niet, omdat ik was vergeten mijn SSH-key op te slaan bij TransIP. Hierdoor kreeg ik steeds een foutmelding dat de toegang geweigerd werd.

Daarna heb ik via het controlepaneel van TransIP alsnog mijn publieke SSH-key toegevoegd aan de VPS-instellingen. Nadat de key was opgeslagen, werkte de verbinding direct wel en kon ik via PowerShell succesvol inloggen op de server.

Hierdoor leerde ik dat SSH alleen werkt wanneer de juiste publieke key correct aan de server is gekoppeld.

Actieve SSH keys

Beschrijving	Fingerprint	Datum	Standaardselectie	Verwijderen
Desktop Amira	78:1b:ad:f3:5e:f6:05:c5:9a:d3:9e:57:03:cf:fe:e3	2026-05-24 23:00:56	Nee	X

```
aelhafidi@cloud: ~  
* Management: https://landscape.canonical.com  
* Support: https://ubuntu.com/pro  
  
System information as of Wed May 27 23:14:06 UTC 2026  
  
System load: 0.07  
Usage of /: 5.3% of 96.73GB  
Memory usage: 55%  
Swap usage: 0%  
Processes: 115  
Users logged in: 0  
IPv4 address for ens3: 37.97.220.7  
IPv6 address for ens3: 2a01:7c8:aac4:10d:5054:ff:fee2:5eed  
  
* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s  
just raised the bar for easy, resilient and secure K8s cluster deployment.  
  
https://ubuntu.com/engage/secure-kubernetes-at-the-edge  
  
Expanded Security Maintenance for Applications is not enabled.  
  
1 update can be applied immediately.  
To see these additional updates run: apt list --upgradable  
  
Enable ESM Apps to receive additional future security updates.  
See https://ubuntu.com/esm or run: sudo pro status  
  
Last login: Tue May 26 16:10:52 2026 from 88.159.220.169  
aelhafidi@cloud:~$
```

Stap 3: Server voorbereiden

Nadat ik succesvol was ingelogd op de server, heb ik Docker geïnstalleerd. Docker gebruik ik om mijn applicatie op de VPS te draaien. Docker biedt op de officiële website een installatiescript aan waarmee Docker wordt gedownload en geïnstalleerd. Hierdoor hoefde ik niet alles handmatig stap voor stap in te stellen, wat het proces veel eenvoudiger maakte. Na het uitvoeren van het script was Docker direct klaar voor gebruik en kon ik verdergaan met de rest van de installatie (Inc, 2026).

```
aelhafidi@cloud: ~  
containerd:  
Version: v2.2.4  
GitCommit: 193637f7ee8ae5f5aa5248f49e7baa3e6164966e  
runc:  
Version: 1.3.5  
GitCommit: v1.3.5-0-g488fc13e  
docker-init:  
Version: 0.19.0  
GitCommit: de40ad0  
  
=====
```

```
To run Docker as a non-privileged user, consider setting up the  
Docker daemon in rootless mode for your user:  
  
    dockerd-rootless-setuptool.sh install  
  
Visit https://docs.docker.com/go/rootless/ to learn about rootless mode.  
  
To run the Docker daemon as a fully privileged service, but granting non-root  
users access, refer to https://docs.docker.com/go/daemon-access/  
  
WARNING: Access to the remote API on a privileged Docker daemon is equivalent  
to root access on the host. Refer to the 'Docker daemon attack surface'  
documentation for details: https://docs.docker.com/go/attack-surface/  
  
=====
```

```
aelhafidi@cloud:~$
```



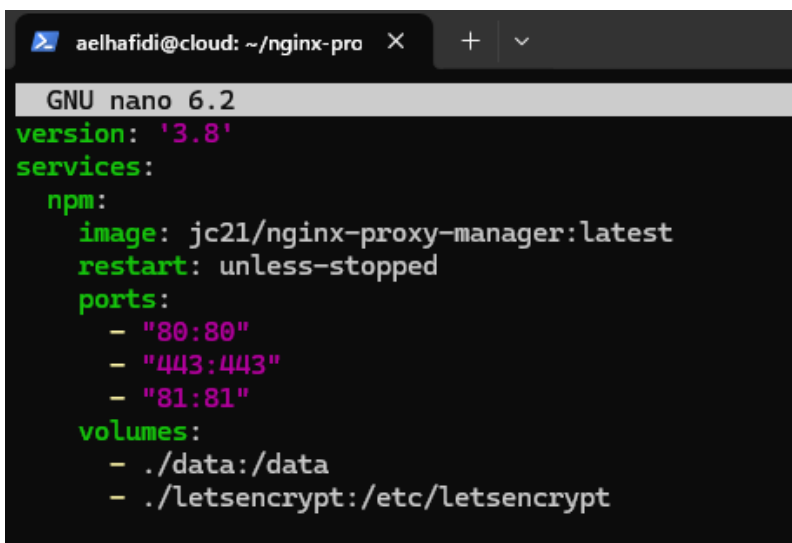
```
aelhafidi@cloud:~$ docker --version
Docker version 29.5.2, build 79eb04c
```

Stap 4: Nginx Proxy Manager installeren

Voor het reverse proxy-gedeelte heb ik gekozen voor Nginx Proxy Manager, ook wel NPM genoemd. Dit programma zorgt ervoor dat bezoekers automatisch naar de juiste applicatie worden doorgestuurd. Daarnaast maakt NPM het eenvoudig om HTTPS in te stellen met let's Encrypt.

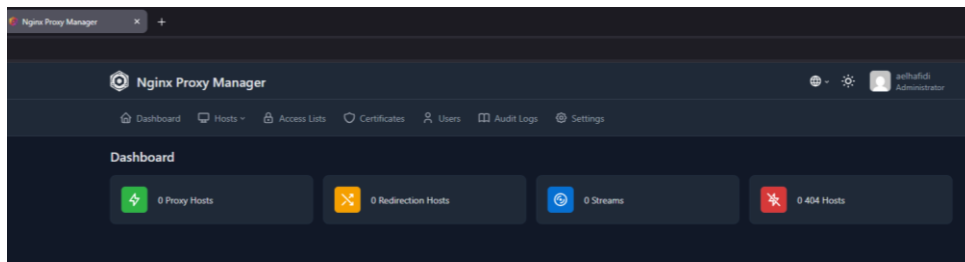
Op de officiële website van Nginx Proxy Manager staat een korte installatiehandleiding waarin precies wordt uitgelegd hoe je NPM via Docker opzet. De handleiding bevat een voorbeeld van een docker-compose bestand met de juiste poorten en instellingen. Dit voorbeeld heb ik gebruikt als basis voor mijn eigen opzet (*Nginx Proxy Manager, z.d.*).

Op de server heb ik een aparte map aangemaakt voor NPM. In deze map heb ik een docker-compose.yml bestand geplaatst, gebaseerd op het voorbeeld van de NPM-website. In dit bestand staat welke NPM-container Docker moet starten en welke poorten gebruikt worden. NPM gebruikt drie poorten: poort 80 voor HTTP-verkeer, poort 443 voor HTTPS-verkeer en poort 81 voor de beheeromgeving.



```
aelhafidi@cloud: ~/nginx-pro X + v
GNU nano 6.2
version: '3.8'
services:
  npm:
    image: jc21/nginx-proxy-manager:latest
    restart: unless-stopped
    ports:
      - "80:80"
      - "443:443"
      - "81:81"
    volumes:
      - ./data:/data
      - ./letsencrypt:/etc/letsencrypt
```

Na het opstarten van Docker kon ik via de browser naar het IP-adres van de server met :81 erachter gaan om in te loggen op de beheeromgeving van NPM. De standaard inloggegevens waren admin@example.com met het wachtwoord changeme. Deze heb ik direct aangepast naar mijn eigen gegevens, omdat de standaard inloggegevens publiek bekend zijn en dus onveilig.



Stap 5: BlockyTest naar de server brengen

Mijn applicatie staat in mijn GitLab-repository. Om de code naar de server te krijgen heb ik git clone gebruikt. Dit is een standaard Git-commando waarmee je een complete repository van GitLab kopieert naar je server.

```
aelhafidi@cloud:~$ git clone https://gitlab.fdmci.hva.nl/propedeuse-hbo-ict/onderooguuuxoo98.git
Cloning into 'leefooguuuxoo98'...
Username for 'https://gitlab.fdmci.hva.nl': HAFIDIA1
Password for 'https://HAFIDIA1@gitlab.fdmci.hva.nl':
remote: Enumerating objects: 7877, done.
remote: Counting objects: 100% (395/395), done.
remote: Compressing objects: 100% (393/393), done.
remote: Total 7877 (delta 215), reused 0 (delta 0), pack-reused 7482 (from 1)
Receiving objects: 100% (7877/7877), 37.60 MiB | 17.88 MiB/s, done.
Resolving deltas: 100% (2490/2490), done.
aelhafidi@cloud:~$
```

Voor de authenticatie heb ik een Personal Access Token gebruikt in plaats van een wachtwoord. De HvA GitLab accepteert geen gewone wachtwoorden meer voor Git-acties, dus moest ik een token aanmaken in mijn GitLab-profiel. Op de officiële GitLab documentatie staat uitgelegd hoe je dit doet en welke rechten je aan een token kunt geven (*Personal Access Tokens | GitLab Docs, z.d.*).

Name	Status	Scopes	Usage ⓘ	Lifetime
VPS	Active	read_repository	Last used: 33 minutes ago IPs: 37.97.220.7, 2a01:7c8:aac4:10d:5054:ff:fee2:5eed	🕒 In 3 weeks 🕒 May 25, 2026

Het project bevatte al een Dockerfile en een docker-compose.yml bestand die ik eerder met mijn team had opgezet. In de Dockerfile staat hoe Docker de container moet bouwen, welke Python-versie gebruikt moet worden en welke dependencies geïnstalleerd moeten worden. Het docker-compose.yml bestand bepaalt hoe de container gestart moet worden.

```

docker-compose.yml X
docker-compose.yml
  Run All Services | You, last month | 2 authors (HBO-ICT Bot and one other)
1  services:
   Run Service
2    dokkie-image:
3      build: .
4      image: blockytest
5      ports:
6        - "5000:5000"
7      container_name: de-opdracht-container
8      env_file:
9        - .env

```

Wat nog ontbrak was een .env bestand met de gegevens voor de databaseverbinding. Dit bestand staat niet in de repository vanwege veiligheidsredenen. Wachtwoorden en API-keys moeten nooit in een Git-repository terechtkomen, omdat iedereen met toegang tot de repository die anders kan zien. Daarom heb ik het .env bestand handmatig aangemaakt op de server met de juiste API-URL, API-key en databasenaam van mijn project op hbo-ict.cloud (Linuxize, 2022).

Daarna kon ik de applicatie starten met het commando docker compose up. Docker bouwde eerst de container op basis van de Dockerfile en startte vervolgens de applicatie op. De Flask-applicatie draaide standaard op poort 5000 binnen de container.

```

=> CACHED [2/5] WORKDIR /app 0.0s
=> CACHED [3/5] COPY requirements.txt . 0.0s
=> CACHED [4/5] RUN pip install -r requirements.txt 0.0s
=> CACHED [5/5] COPY . . 0.0s
=> exporting to image 0.2s
=> exporting layers 0.0s
=> exporting manifest sha256:7add51aefafcc8e4b764f41477673e89dce5a9651ab757da5e567f1993 0.0s
=> exporting config sha256:ce08274851e97066cea2279dbc79eddc04d6bc93211c163580ef491a9afa 0.0s
=> exporting attestation manifest sha256:579ab8bb01a6f383e19102eb5258ac588cea0002112520 0.1s
=> exporting manifest list sha256:9391060f7356bbabd4496f25417f099295f6605425103b28b069 0.0s
=> naming to docker.io/library/blockytest:latest 0.0s
=> unpacking to docker.io/library/blockytest:latest 0.0s
=> resolving provenance for metadata file 0.0s
[+] up 2/2
Image blockytest Built 1.7s
Container de-opdracht-container Started 0.5s
aelhafidi@cloud:~/nginx-proxy-manager/leefooguuxoo98$

```

Hoewel Nginx Proxy Manager en BlockyTest allebei succesvol draaiden op de server, werkte de verbinding tussen de twee eerst nog niet. Dit kwam doordat beide applicaties een eigen docker-compose.yml bestand hadden en daardoor automatisch in een apart Docker-netwerk draaiden. Containers in verschillende netwerken kunnen niet direct met elkaar communiceren.

Om dit op te lossen heb ik ervoor gezorgd dat beide containers in hetzelfde Docker-netwerk terechtkwamen. Hierdoor konden Nginx Proxy Manager en mijn Flask-applicatie elkaar wel bereiken.

```

Last login: Wed May 27 23:46:03 2026 from 88.159.220.169
aelhafidi@cloud:~$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
77e6eda5168b   blockytest                          "flask run --host 0.0.0.0" 35 hours ago   Up 35 hours   5000/tcp
4c701402d6f8   jc21/nginx-proxy-manager:latest    "/init"                  36 hours ago   Up 36 hours   0.0.0.0:80-81->80-81/tcp, [::]:80-81->[::]:80-81/tcp, 0.0.0.0:443->443/tcp, [::]:443->[::]:443/tcp
nginx-proxy-manager-npm-1

```

Met het commando `docker ps` heb ik gecontroleerd welke containers actief draaien op de server. Hier is te zien dat zowel mijn Flask-applicatie als Nginx Proxy Manager actief zijn.

Stap 6: Domeinnaam `blockytest.nl` koppelen

De domeinnaam `blockytest.nl` had ik al gekocht bij TransIP voor ongeveer 59 cent voor het eerste jaar. Om de domeinnaam te koppelen aan mijn VPS moest ik een DNS A-record aanmaken. Een A-record zorgt ervoor dat een domeinnaam verwijst naar het IP-adres van een server.

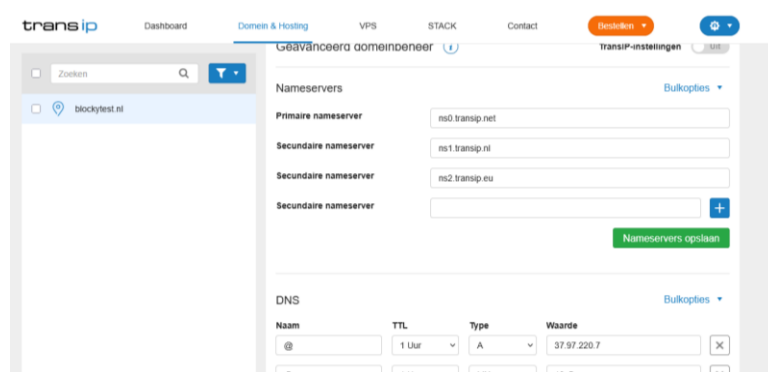
/ Factuur

Omschrijving	Termijn	Factureringsperiode	Aantal	Prijs
Domeinregistratie .nl (<code>blockytest.nl</code>)	Jaarlijks	24-05-2026 - 23-05-2027	1	€ 16,50
+ Korting (Eerste jaar .nl voor € 0,49)	Eenmalig	24-05-2026 - 23-05-2027	1	€ -16,01
Subtotaal (zonder korting)				€ 16,50
- Korting				€ -16,01
Subtotaal				€ 0,49
+ BTW (21%)				€ 0,10
Totaal:				€ 0,59

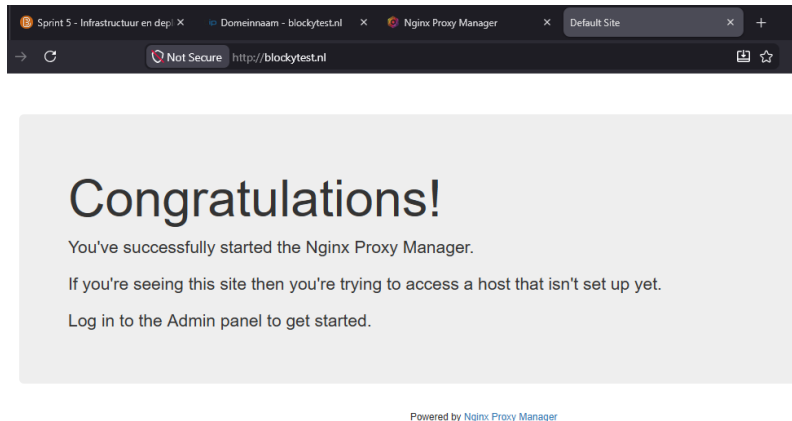
Ik wist in het begin niet precies hoe DNS-records werkten en waar ik die moest instellen. Via de knowledge base van TransIP vond ik een artikel dat uitlegt wat DNS-records zijn en hoe je ze beheert in het controlepaneel. Daar las ik dat er verschillende soorten records bestaan zoals A, AAAA, en dat een A-record specifiek wordt gebruikt om een domeinnaam te koppelen aan een IPv4-adres (*De DNS-instellingen van Mijn Webhostingpakket - TransIP, z.d.*).

In het controlepaneel van TransIP ging ik naar de DNS-instellingen van `blockytest.nl`. Daar zag ik dat “TransIP-beheer” nog aan stond. Hierdoor kon ik zelf geen DNS-records aanpassen. Daarom heb ik deze instelling uitgezet, zodat ik handmatig wijzigingen kon maken.

Daarna heb ik een A-record toegevoegd met het IP-adres `37.97.220.7`, het adres van mijn VPS. Hierdoor wist het internet dat bezoekers van `blockytest.nl` naar mijn server gestuurd moesten worden.



Na het opslaan duurde het een paar minuten voordat de wijziging actief was. Toen ik daarna blockytest.nl in mijn browser opende, kreeg ik een melding van Nginx Proxy Manager dat er nog geen host was ingesteld. Dit liet zien dat de domeinnaam correct was gekoppeld aan mijn VPS en dat het verkeer succesvol bij de server aankwam.

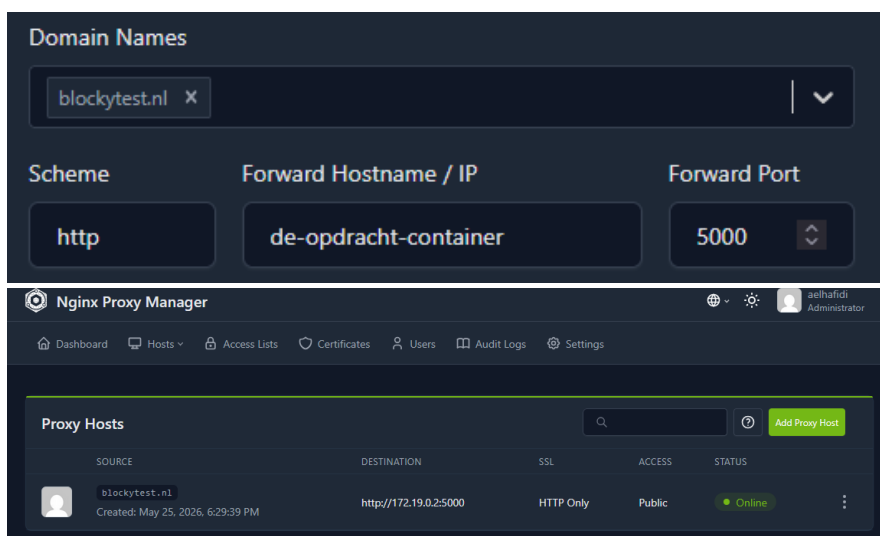


Stap 7: Reverse proxy configureren in NPM

In Nginx Proxy Manager heb ik een nieuwe Proxy Host aangemaakt. Bij *Domain Names* vulde ik blockytest.nl in. Daarna moest ik bij *Forward Hostname* aangeven naar welke container het verkeer doorgestuurd moest worden.

Tijdens het uitzoeken van de juiste instelling kwam ik erachter dat Docker containers binnen een netwerk met elkaar laat communiceren via hun containernaam. Daarom heb ik uiteindelijk de containernaam de-opdracht-container gebruikt als *Forward Hostname* in plaats van een IP-adres.

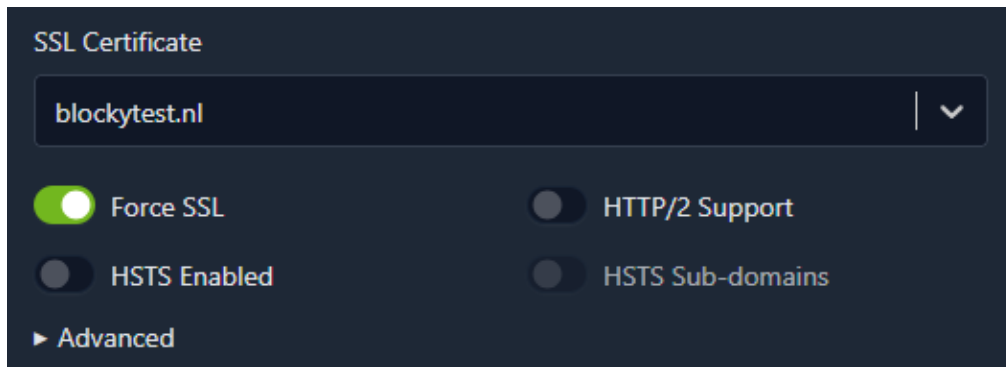
Dit is betrouwbaarder, omdat het IP-adres van een container kan veranderen wanneer de container opnieuw wordt gestart. De containernaam blijft hetzelfde en Docker koppelt deze automatisch aan het juiste IP-adres binnen het netwerk (Inc, 2026a).



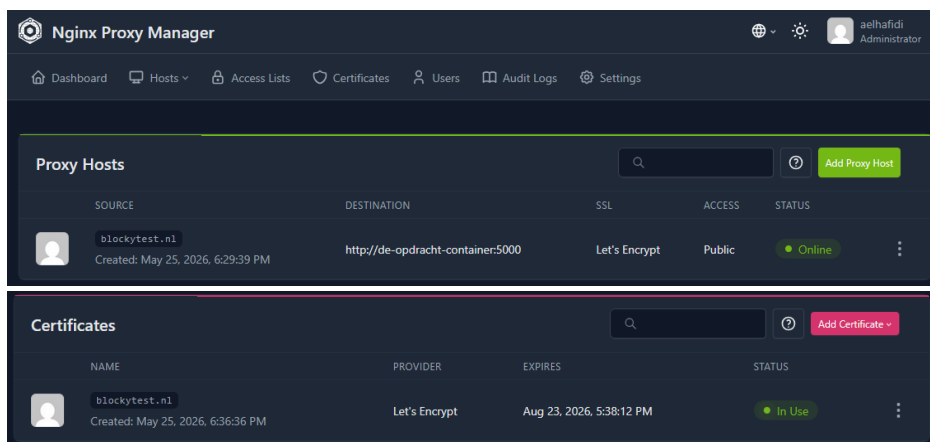
Stap 8: HTTPS instellen met Let's Encrypt

Een website zonder HTTPS is niet veilig en moderne browsers waarschuwen daarvoor. Gelukkig biedt NPM een ingebouwde optie om gratis SSL-certificaten aan te vragen via Let's Encrypt.

In de instellingen van de Proxy Host ging ik naar het SSL-tabblad. Daar koos ik voor Request a new SSL Certificate. Ik vulde mijn e-mailadres in, ging akkoord met de voorwaarden van Let's Encrypt en zette Force SSL aan. Force SSL zorgt ervoor dat alle bezoekers automatisch worden doorgestuurd naar de https-versie.



Na een paar seconden was het SSL-certificaat aangemaakt en automatisch gekoppeld aan blockytest.nl. Vanaf dat moment was de website veilig bereikbaar via HTTPS en verscheen het slotje in de browser.



7. CI/CD pipeline opzetten

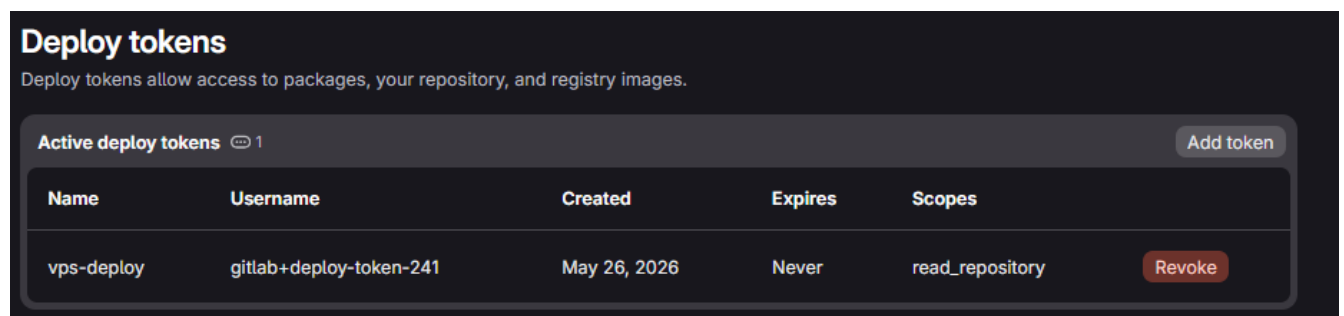
Op dit moment was de applicatie online, maar elke keer als ik een wijziging in mijn code maakte moest ik handmatig inloggen op de server, git pull doen en de container opnieuw bouwen. Daarom heb ik een CI/CD pipeline opgezet die dit automatisch doet bij elke wijziging op de main branch.

Voor het opzetten van de CI/CD-pipeline heb ik gebruikgemaakt van een uitlegvideo van Programonaut op YouTube. Deze video hielp mij bij het begrijpen van hoe GitLab CI/CD werkt en hoe je automatisch kunt deployen via SSH (*Programonaut, 2023*).

Stap 1: Aparte SSH key voor deployment

Voor de pipeline heb ik een aparte SSH key aangemaakt. Het is niet veilig om je persoonlijke SSH key te gebruiken voor automatische deployment, want dan zou GitLab toegang hebben tot al je servers waar die key voor gebruikt wordt. Een aparte deploy key kan ik direct intrekken als er ooit iets fout gaat.

De publieke key heb ik toegevoegd aan de server, op dezelfde manier als ik mijn persoonlijke key had toegevoegd. De private key bewaarde ik om door te geven aan GitLab.



Stap 2: Variabelen instellen in GitLab

In GitLab kun je geheime gegevens opslaan als CI/CD variabelen. Deze variabelen worden niet getoond in logs en blijven veilig opgeslagen. Ik heb drie variabelen aangemaakt.

De eerste variabele was SSH_PRIVATE_KEY met de inhoud van mijn nieuwe private key. Bij het aanmaken kreeg ik eerst een foutmelding omdat ik had aangevinkt dat de variabele gemaskeerd moest worden. Gemaskeerde variabelen mogen geen spaties of nieuwe regels bevatten, maar een SSH key heeft dat juist wel. Ik heb de masking uitgezet en daarna werkte het.

De tweede variabele was SERVER_HOST met het IP-adres van mijn VPS, 37.97.220.7. De derde was SERVER_USER met mijn gebruikersnaam aelhafidi.

CI/CD Variables </> 3		Reveal values	Add variable
Key ↑	Value	Environments	Actions
SERVER_HOST		All (default)	 
SERVER_USER		All (default)	 
SSH_PRIVATE_KEY		All (default)	 

Stap 3: Het pipeline configuratiebestand

In GitLab gebruik je een bestand met de naam `.gitlab-ci.yml` om te bepalen wat de CI/CD-pipeline moet doen. Dit bestand stond al in mijn project. Ik heb hier mijn eigen deploy-stappen aan toegevoegd.

```

deploy_to_vps:
  stage: deploy
  image: alpine:latest
  tags:
    - hva
  before_script:
    # SSH setup voor verbinding met de VPS
    - apk add --no-cache openssh-client
    - mkdir -p ~/.ssh
    - echo "$SSH_PRIVATE_KEY" > ~/.ssh/id_ed25519
    - chmod 600 ~/.ssh/id_ed25519
    - ssh-keyscan -H $SERVER_HOST >> ~/.ssh/known_hosts
  script:
    # Log in op de server, haal nieuwe code op en herstart de app
    - ssh $SERVER_USER@$SERVER_HOST "cd ~/nginx-proxy-manager/leefoc
  only:
    - main

```

De deploy job werkt als volgt. Wanneer ik nieuwe code naar de main branch push, start GitLab automatisch een tijdelijke omgeving om de pipeline uit te voeren. Vanuit deze omgeving maakt GitLab via SSH verbinding met mijn VPS-server.

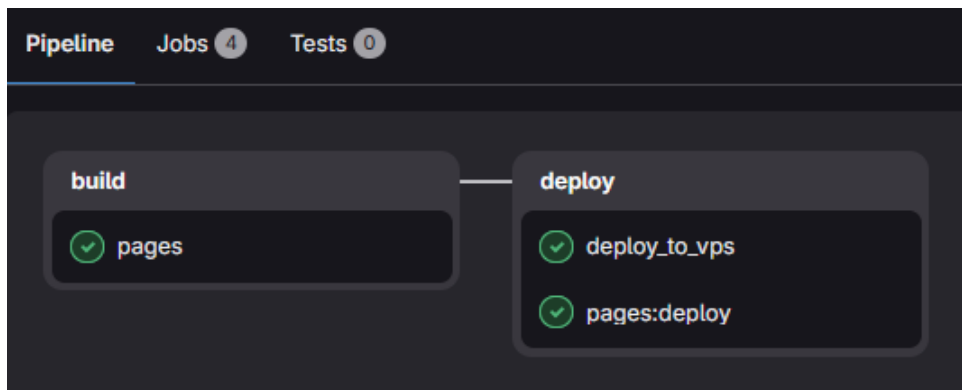
Daarna voert de pipeline automatisch een git pull uit op de server om de nieuwste versie van de code op te halen. Vervolgens wordt docker compose opnieuw gestart zodat de applicatie bijgewerkt wordt met de nieuwe code.

De pipeline draait alleen op de main branch. Hierdoor worden alleen definitieve wijzigingen automatisch gedeployed en niet iedere wijziging van andere branches.

Stap 4: De HvA tag

Bij de eerste run van de pipeline bleef de deploy job hangen op "pending". Er was geen GitLab Runner beschikbaar om de job uit te voeren. Na wat zoeken kwam ik erachter dat de HvA hun eigen GitLab Runners heeft die alleen jobs oppakken met de tag hva.

Ik heb deze tag toegevoegd aan de deploy job en daarna pakte de HvA Runner de job direct op. De pipeline werkte en mijn applicatie werd automatisch gedeployed.



8. Hoe het allemaal samenwerkt

Wanneer een bezoeker naar <https://blockytest.nl> gaat, gebeurt het volgende achter de schermen.

De browser vraagt eerst aan een DNS server welk IP-adres bij blockytest.nl hoort. Door mijn A record in TransIP weet DNS dat het bij 37.97.220.7 hoort. De browser maakt verbinding met die server.

Op die server draait Nginx Proxy Manager en die ontvangt het verzoek. NPM kijkt naar welke domeinnaam de bezoeker heeft opgevraagd en stuurt het verzoek door naar de juiste container, in dit geval de-opdracht-container op poort 5000. Tegelijkertijd zorgt NPM voor de versleuteling met het Let's Encrypt certificaat.

De Flask applicatie verwerkt het verzoek, haalt eventueel data op uit de database en stuurt het antwoord terug. NPM versleutelt het antwoord en stuurt het naar de browser. De browser toont de pagina met het slotje.

Bij elke nieuwe code push naar main gebeurt automatisch het volgende. GitLab detecteert de push en start de pipeline. De HvA GitLab Runner pakt de deploy job op. De pipeline maakt SSH-verbinding met de VPS. Op de server wordt git pull uitgevoerd om de nieuwe code op te halen en docker compose up bouwt de container opnieuw. De oude container wordt vervangen door de nieuwe en de site is direct bij met de wijzigingen.

→ ↻ blockytest.nl/projects/login

WailSalutem Foundation **BlockyTest**

Inloggen

Gebruikersnaam of e-mailadres:

Wachtwoord:

[Inloggen](#) [Account aanmaken](#)

← → ↻ blockytest.nl/blocky/?project_id=45

WailSalutem Foundation **BlockyTest** [Mijn projecten](#) [Opslaan](#) [Laden](#) [Run](#) [Clear](#) [Geschiedenis](#)

test

- Structuur
- Browser
- Acties
- Controleren
- Waarden

Gegenereerde code [Kopie](#)

```
# Connect blocks to generate code...
```

Test resultaten

Klik op "Run" om de test uit te voeren

9. Reflectie en geleerde lessen

Tijdens deze opdracht heb ik geleerd hoe verschillende onderdelen van een deploy stack samenwerken, zoals DNS, reverse proxies, Docker en CI/CD. In het begin wist ik alleen hoe ik mijn Flask-app lokaal kon starten, maar nu begrijp ik beter hoe een applicatie online wordt gezet en bereikbaar wordt gemaakt via een server en domeinnaam.

Hieronder beschrijf ik de belangrijkste technische dingen die ik tijdens deze opdracht heb geleerd aan de hand van concrete voorbeelden:

- Ik weet nu dat Docker-containers binnen een netwerk met elkaar communiceren via hun containernaam. Daarom heb ik in Nginx Proxy Manager niet het IP-adres gebruikt, maar de containernaam de-opdracht-container als *Forward Hostname*. Dit werkt beter omdat IP-adressen van containers kunnen veranderen wanneer een container opnieuw start, terwijl de naam hetzelfde blijft.
- Ik weet nu hoe DNS-records werken en hoe een domeinnaam gekoppeld wordt aan een server. Hiervoor heb ik een A-record ingesteld dat blockytest.nl verwijst naar het IP-adres van mijn VPS. Ook heb ik geleerd dat DNS-wijzigingen soms niet direct zichtbaar zijn doordat oude gegevens tijdelijk opgeslagen blijven.
- Ik weet nu hoe CI/CD-variabelen in GitLab werken. Bij het toevoegen van mijn SSH-key kreeg ik eerst een foutmelding omdat ik de optie *masked* had aangezet. Ik heb geleerd dat een SSH-key meerdere regels bevat en daardoor niet als gemaskeerde variabele opgeslagen kan worden. Nadat ik dit had aangepast werkte de pipeline wel.
- Ik weet nu hoe SSH-authenticatie werkt met een publieke en private key. In het begin lukte het inloggen op mijn VPS niet omdat ik vergeten was mijn publieke key toe te voegen aan de server. Nadat ik dit had gedaan kon ik wel veilig inloggen via SSH zonder wachtwoord.
- Ik weet nu hoe HTTPS ingesteld wordt met Let's Encrypt in Nginx Proxy Manager. Door de optie *Force SSL* aan te zetten worden bezoekers automatisch doorgestuurd naar de beveiligde HTTPS-versie van de website. Ook heb ik geleerd dat SSL-certificaten automatisch vernieuwd kunnen worden.
- Ik weet nu dat GitLab Runners werken met tags. Mijn pipeline bleef eerst hangen op *pending*, omdat er geen juiste runner gevonden werd. Nadat ik de tag *hva* had toegevoegd in mijn `.gitlab-ci.yml` bestand, werd de pipeline wel uitgevoerd en

kon GitLab automatisch de nieuwste versie van mijn applicatie deployen naar de VPS.

10. Conclusie

De applicatie BlockyTest is succesvol gedeployed op <https://blockytest.nl> via een eigen VPS bij TransIP. De deploy stack bestaat uit Docker containers, Nginx Proxy Manager als reverse proxy en een geautomatiseerde CI/CD pipeline via GitLab van de HvA. De applicatie is bereikbaar via HTTPS met een gratis Let's Encrypt certificaat dat zichzelf automatisch vernieuwt.

Mijn route naar deze oplossing was niet rechtlijnig. Ik heb verschillende hosting opties overwogen, ben tegen technische problemen aangelopen en heb veel moeten experimenteren. Achteraf gezien zijn juist die problemen het meest leerzaam geweest. Ik begrijp nu niet alleen hoe je iets deployt, maar ook waarom bepaalde keuzes gemaakt worden en wat er achter de schermen gebeurt.

11. Literatuurlijst

- Inc, D. (2026, 23 april). *Install Docker Engine on Ubuntu*. Docker Documentation.
<https://docs.docker.com/engine/install/ubuntu/>
- *Nginx Proxy Manager*. (z.d.). <https://nginxproxymanager.com/setup/>
- *Personal access tokens | GitLab Docs*. (z.d.). GitLab Docs.
https://docs.gitlab.com/user/profile/personal_access_tokens/
- Linuxize (2022). How to Use Nano Text Editor. <https://linuxize.com/post/how-to-use-nano-text-editor/>
- *De DNS-instellingen van mijn webhostingpakket - TransIP*. (z.d.).
<https://www.transip.nl/knowledgebase/527-de-dns-instellingen-van-mijn-webhostingpakket>
- Inc, D. (2026a, maart 17). *Networking overview*. Docker Documentation.
<https://docs.docker.com/engine/network/>
- Programonaut. (2023, 4 juni). *How to deploy a GIT repository to a server using GitLab CI/CD* [Video]. YouTube.
<https://www.youtube.com/watch?v=JovaNZyFhLI>

12. AI-gebruik tijdens deze opdracht

Tijdens deze opdracht heb ik beperkt gebruikgemaakt van AI als ondersteuning bij het leren en oplossen van problemen. Ik heb AI vooral gebruikt om nieuwe begrippen beter te begrijpen, zoals DNS, reverse proxies en Docker-netwerken.

Ook heb ik AI soms gebruikt om foutmeldingen beter te begrijpen, zodat ik gericht kon zoeken naar een oplossing in de officiële documentatie.

Daarnaast heb ik AI gebruikt om mijn eigen teksten te controleren op duidelijkheid en taalgebruik. De inhoud, keuzes en uitgevoerde stappen in dit verslag heb ik zelf gedaan en gebaseerd op mijn eigen ervaringen tijdens het deployen van de applicatie.

De belangrijkste bronnen voor deze opdracht waren de officiële documentatie van Docker, GitLab, TransIP en Nginx Proxy Manager. AI is alleen gebruikt als extra hulpmiddel naast deze bronnen.